SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks: 30**

Annexure A

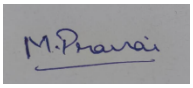
Dry Run Evaluation

Name / Reg No: M.Pranai (192525068)

Code of Conduct:

I M.Pranai (192525068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

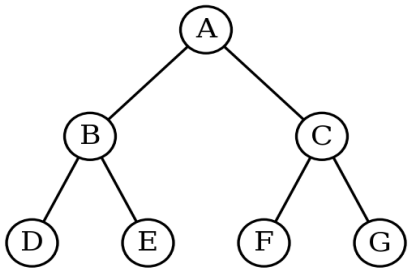
Signature of the student: 

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS) (10 Marks)

Consider the following graph:



Task: Starting from node A, perform a Breadth-First Search (BFS). Complete the following table.

Solution (FIFO queue; children added left to right):

Iteration	Queue (Before Expansion)	Visited Node	Queue (After Expansion)
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

Questions:

1. Write the BFS traversal order.

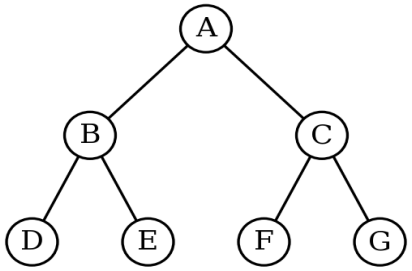
Answer: A → B → C → D → E → F → G

2. Show the queue contents after each iteration.

Answer: Start: [A]. After iteration 1: [B, C]; 2: [C, D, E]; 3: [D, E, F, G]; 4: [E, F, G]; 5: [F, G]; 6: [G]; 7: [ ] (empty).

2. Depth-First Search (DFS) (10 Marks)

Using the same graph:



Task: Perform Depth-First Search (DFS) starting from A. Complete the table.

Solution (LIFO stack, top of stack written on the left; right child pushed first so the left child is explored first):

Iteration	Stack (Before Pop)	Visited Node	Stack (After Pushing Children)
1	A	A	B, C
2	B, C	B	D, E, C
3	D, E, C	D	E, C
4	E, C	E	C
5	C	C	F, G
6	F, G	F	G
7	G	G	Empty

DFS traversal order: A → B → D → E → C → F → G

Explanation: DFS goes as deep as possible along one branch (A → B → D) before backtracking to explore the remaining branches (E, then C and its subtree F, G).

3. 8-Puzzle Problem using BFS (10 Marks)

Initial State :	**Goal State :**
1 3 6 5 0 2 4 7 8	1 2 3 4 5 6 7 8 0

Task: Trace Breadth-First Search (BFS) until the goal state is reached. Fill in the table.

Assumption: the blank (0) is moved in the order Up, Down, Left, Right, and states already generated are not generated again. States are labelled S0, S1, S2, … in the order they are generated (see the state key below the table).

Iteration	**Current State**	**Queue Before Expansion**	**Queue After Expansion**
1	S0 1 3 6 5 0 2 4 7 8	S0	S1, S2, S3, S4
2	S1 1 0 6 5 3 2 4 7 8	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	S2 1 3 6 5 7 2 4 0 8	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	S3 1 3 6 0 5 2 4 7 8	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	S4 1 3 6 5 2 0 4 7 8	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

State key (first five expansions):

Label	**State (row / row / row)**	**Generated from**	**Blank moved**
S0	1 3 6 / 5 0 2 / 4 7 8	Initial state	–
S1	1 0 6 / 5 3 2 / 4 7 8	S0	Up
S2	1 3 6 / 5 7 2 / 4 0 8	S0	Down
S3	1 3 6 / 0 5 2 / 4 7 8	S0	Left
S4	1 3 6 / 5 2 0 / 4 7 8	S0	Right
S5	0 1 6 / 5 3 2 / 4 7 8	S1	Left

Label	State (row / row / row)	Generated from	Blank moved
S6	1 6 0 / 5 3 2 / 4 7 8	S1	Right
S7	1 3 6 / 5 7 2 / 0 4 8	S2	Left
S8	1 3 6 / 5 7 2 / 4 8 0	S2	Right
S9	0 3 6 / 1 5 2 / 4 7 8	S3	Up
S10	1 3 6 / 4 5 2 / 0 7 8	S3	Down
S11	1 3 0 / 5 2 6 / 4 7 8	S4	Up
S12	1 3 6 / 5 2 8 / 4 7 0	S4	Down

Questions

1. Which node is expanded in each iteration?

BFS expands nodes in the order they were generated (first-in, first-out): iteration 1 expands S0 (initial state), iteration 2 expands S1, iteration 3 expands S2, iteration 4 expands S3, iteration 5 expands S4, and so on, level by level. The goal state is at depth 8, so BFS must finish expanding all nodes of depths 0 to 7 before it is dequeued as the 311th node.

2. How many states are generated in total?

By the time the goal state is removed from the queue, 500 distinct states have been generated (the initial state plus 499 new states), of which 310 were expanded before the goal was selected. Repeated states are discarded.

3. Write the complete solution path from the initial state to the goal state.

The solution has 8 moves (path cost = 8):

Right → Up → Left → Down → Left → Down → Right → Right (moves of the blank tile)

Initial	Step 1: Right	Step 2: Up	Step 3: Left	Step 4: Down	Step 5: Left	Step 6: Down	Step 7: Right	Step 8: Right
1 3 6 5 0 2 4 7 8	1 3 6 5 2 0 4 7 8	1 3 0 5 2 6 4 7 8	1 0 3 5 2 6 4 7 8	1 2 3 5 0 6 4 7 8	1 2 3 0 5 6 4 7 8	1 2 3 4 5 6 0 7 8	1 2 3 4 5 6 7 0 8	1 2 3 4 5 6 7 8 0

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

In the 8-puzzle every move has the same cost (1), so the cost of a path equals its number of moves, i.e. the depth of the node. BFS expands all nodes at depth d before any node at depth d + 1. Therefore the first time the goal state is reached, it is reached at the smallest possible depth, and no shorter path can exist. BFS is also complete because the branching factor is finite (at most 4). The drawback is its high time and memory cost, which grows as O(b^d).